Project 4 — Firewall Evasion Test

Network Scanning and Enumeration — Final Project Report

Student Name	Steve
Project	Project 4 — Firewall Evasion Test
Specialization	Network Scanning and Enumeration
Scanner Machine	Kali Linux (hostname: V2310)
Target Machine	Metasploitable 3 — Ubuntu 14.04 LTS VM — 192.168.10.91
Tools Used	Nmap (on Kali), iptables (on the target)

1. Objective

The objective of this project is to test whether common Nmap scan-evasion techniques can defeat a firewall. A Linux target running iptables (Metasploitable 3, 192.168.10.91) is scanned from Kali Linux first with a normal baseline scan and then with three evasion techniques — packet fragmentation, decoy addresses and slow timing. Each technique is recorded as "passed" or "stopped" in a test matrix. Finally, new hardening rules are added to the target's firewall and a repeat scan proves that they work.

2. Tools Used

Tool	Used For
Nmap 7.991 (Kali Linux)	Baseline scan and the three evasion scans (fragmentation -f, decoys -D, timing templates -T1 / -T2)
iptables v1.4.21 (target)	The target's firewall: the existing ruleset was reviewed and three hardening rules were added with the recent module
Metasploitable 3 (Ubuntu 14.04 LTS)	The Linux target virtual machine whose services and firewall were tested

3. Procedure

Step 1 — Find the target's IP address (on the target).

The target VM (Metasploitable 3) was powered on and logged in as user vagrant. Its network address was read with ifconfig so that Kali knew which machine to scan.

```
ifconfig
```

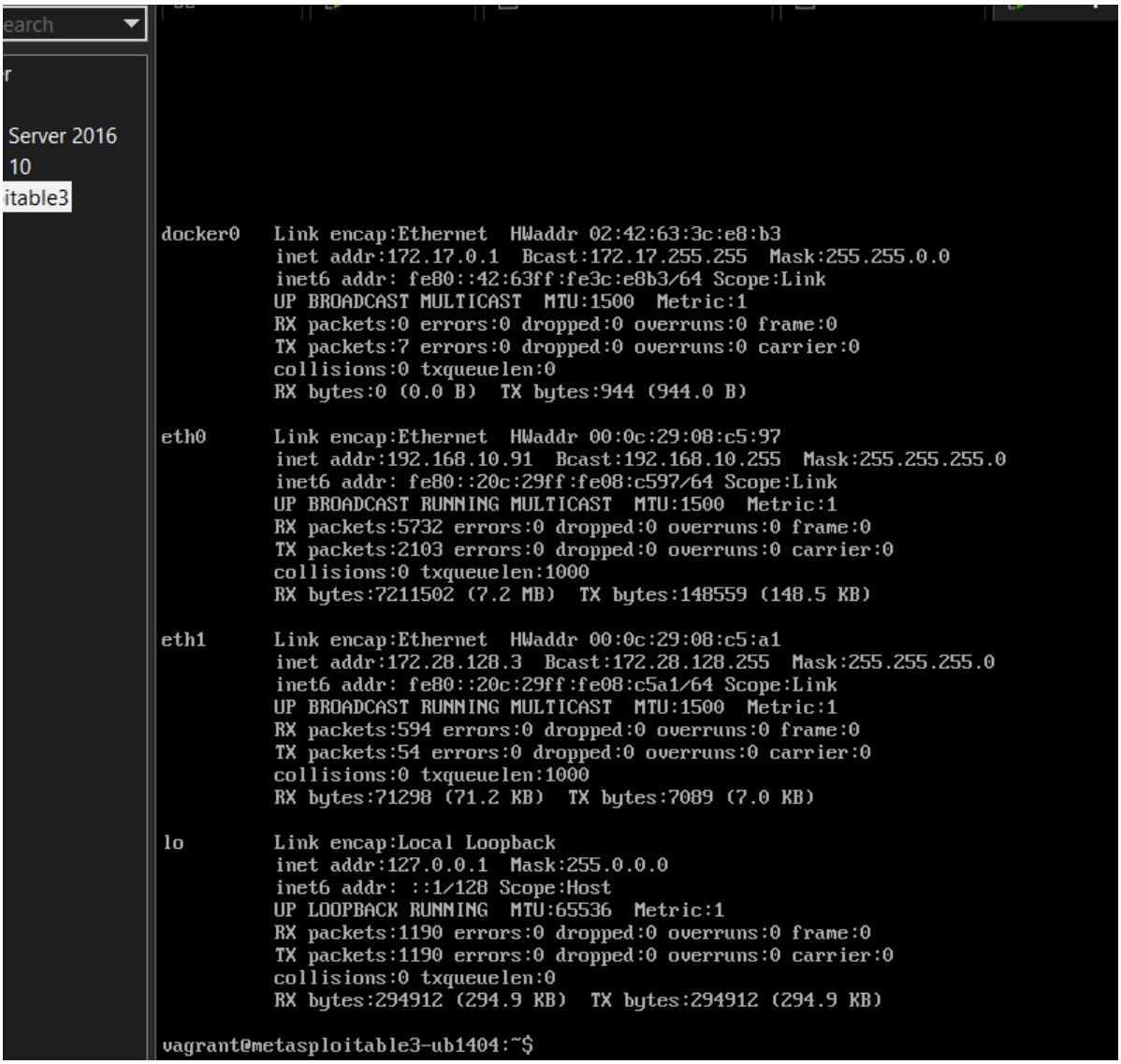


Figure 1 — Step 1: Network interfaces of the target (ifconfig)

What this screenshot shows: The eth0 interface of the target has the address 192.168.10.91 on the lab network (the same network as Kali), with MAC address 00:0C:29:08:C5:97. The other interfaces (eth1, docker0, lo) are internal and are not used. This address, 192.168.10.91, is the target of every scan in this project.

Step 2 — Baseline scan (on Kali).

Before testing any trick, a normal scan recorded what the target looks like through its existing firewall. This baseline is the reference that every evasion scan is compared against.

```
nmap 192.168.10.91 -oN p4-baseline.txt
```

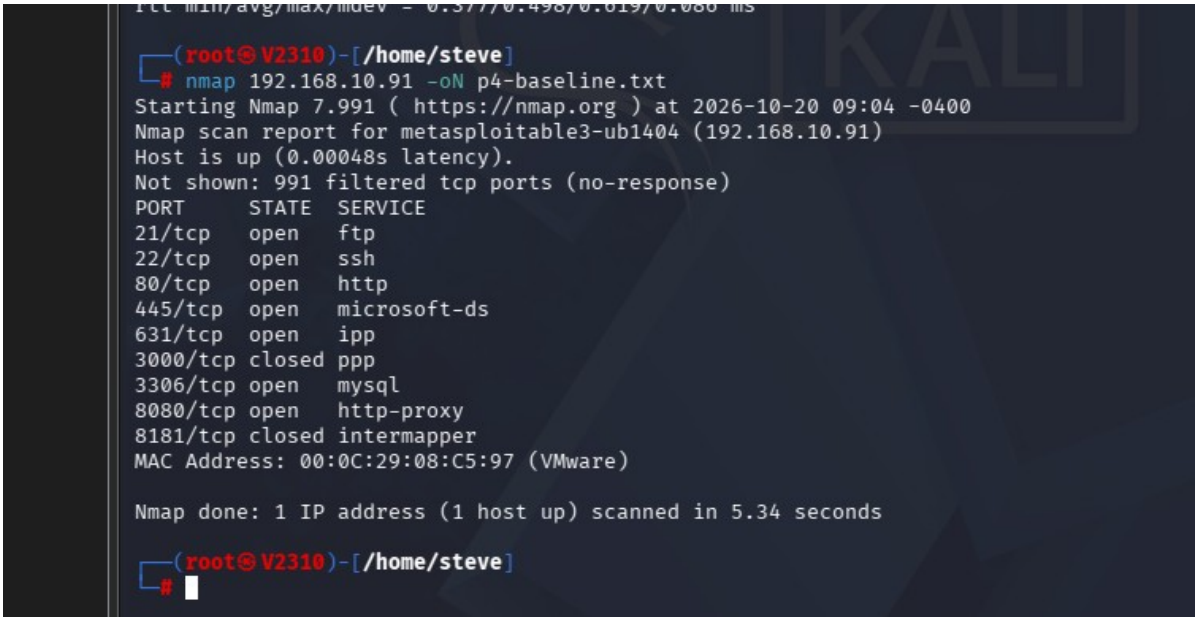


Figure 2 — Step 2: Baseline scan of 192.168.10.91 (no evasion)

What this screenshot shows: The baseline scan found 7 open TCP ports — 21 (FTP), 22 (SSH), 80 (HTTP), 445 (microsoft-ds), 631 (IPP), 3306 (MySQL) and 8080 (http-proxy) — plus 2 closed ports (3000, 8181). The remaining 991 ports are reported as filtered (no-response), which already shows that a firewall is silently dropping most probes. The scan took 5.34 seconds.

Step 3 — Review the existing firewall rules (on the target).

The target's iptables rules were listed to understand what the scans were up against.

```
sudo iptables -L -n --line-numbers
```

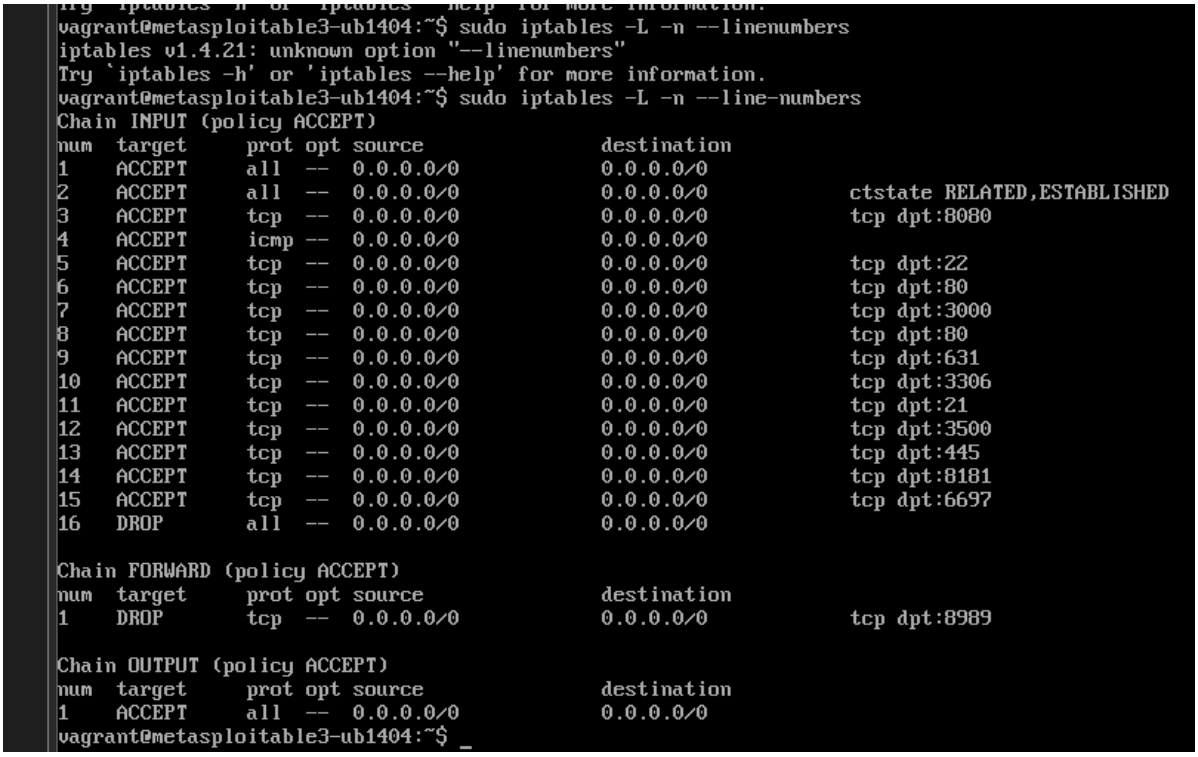


Figure 3 — Step 3: The target's existing iptables INPUT chain

What this screenshot shows: The INPUT chain is an allow-list: after accepting the loopback interface and already-established connections (ctstate RELATED,ESTABLISHED), it accepts ICMP and only named TCP ports (including 8080, 22, 80, 3000, 631, 3306, 21, 3500, 445, 8181 and 6697). The final rule (16) drops everything else, which is why 991 ports showed as filtered in the baseline scan. The FORWARD chain drops TCP port 8989 and the OUTPUT chain accepts all traffic.

Step 4 — Evasion test 1: fragmentation (on Kali).

The scan was repeated with the -f option, which chops each probe into tiny 8-byte fragments. Weak firewalls that cannot reassemble the pieces may let them through and reveal extra ports.

```
sudo nmap -f 192.168.10.91 -oN p4-frag.txt
```

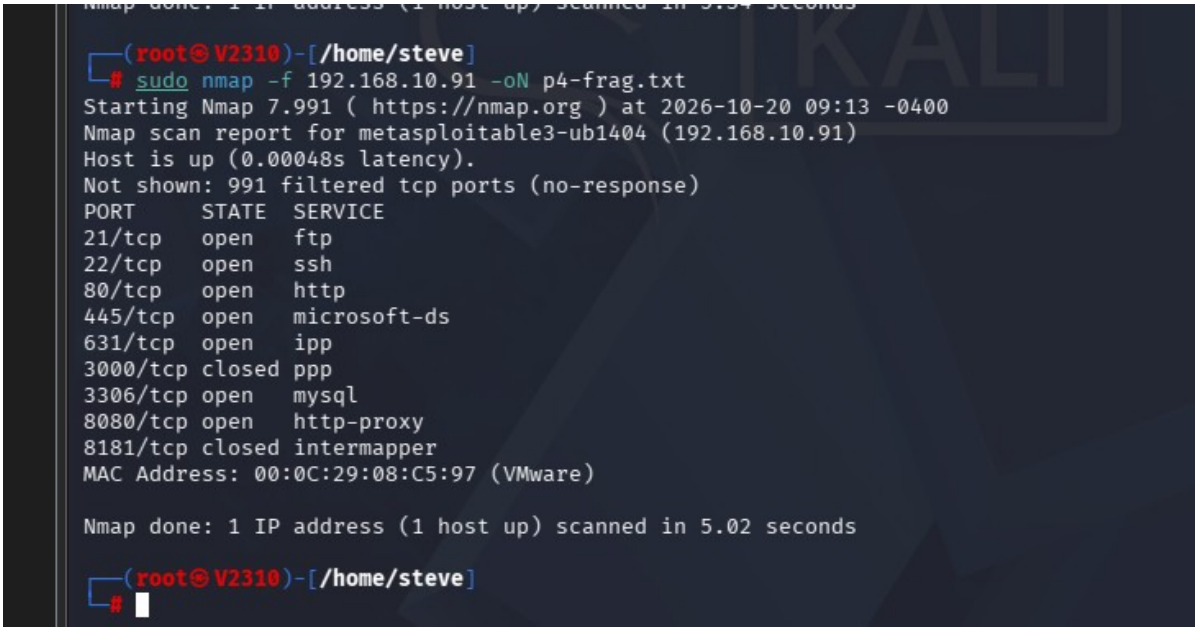


Figure 4 — Step 4: Fragmentation scan (-f)

What this screenshot shows: The fragmentation scan produced exactly the same result as the baseline: the same 7 open ports, the same 2 closed ports and the same 991 filtered ports, in 5.02 seconds. No additional port was revealed. The target's stateful firewall reassembles fragmented packets before applying its rules, so this technique was stopped.

Step 5 — Evasion test 2: decoys (on Kali).

The scan was repeated with decoys, so the target appears to be scanned from five random fake addresses plus the real Kali address at the same time.

```
sudo nmap -D RND:5 192.168.10.91 -oN p4-decoy.txt
```

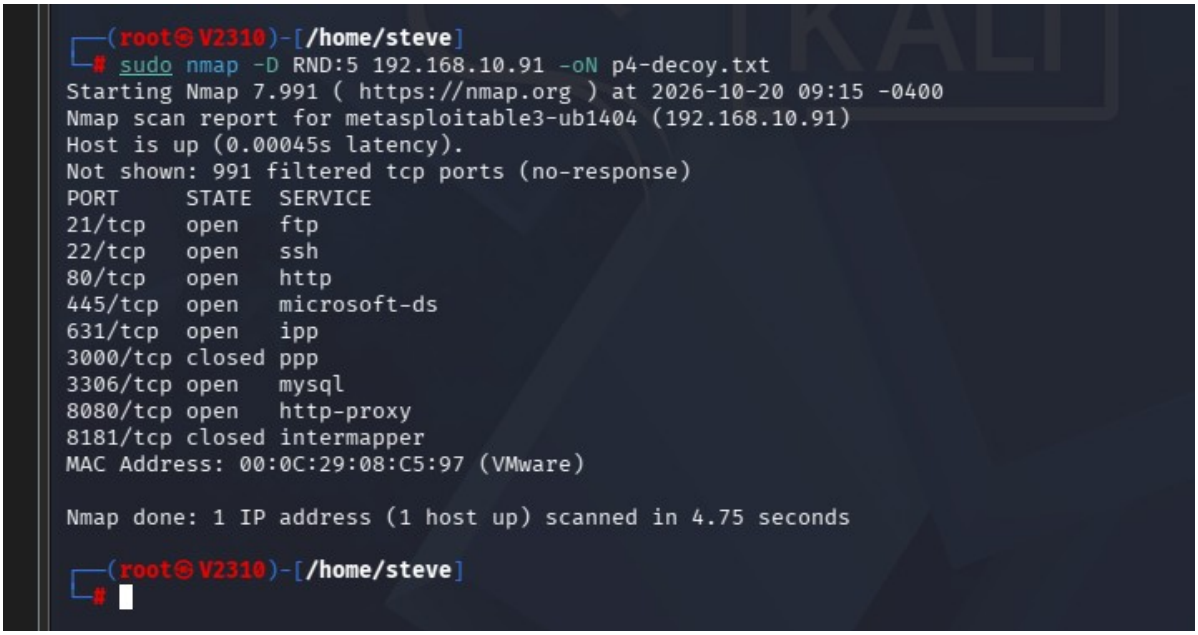


Figure 5 — Step 5: Decoy scan (-D RND:5)

What this screenshot shows: The decoy scan also returned exactly the baseline result — 7 open, 2 closed, 991 filtered — in 4.75 seconds. Decoys hide who is scanning (the firewall log would show six source addresses), but they do not change what the firewall allows, because the rules are port-based, not address-based. No port was bypassed.

Step 6 — Evasion test 3: slow timing (on Kali).

The slowest timing template, -T1 (sneaky), spreads the probes far apart so that rate-based detection does not notice a scan. It was tried first on the top 100 ports.

```
sudo nmap -T1 --top-ports 100 192.168.10.91 -oN p4-timing.txt
```

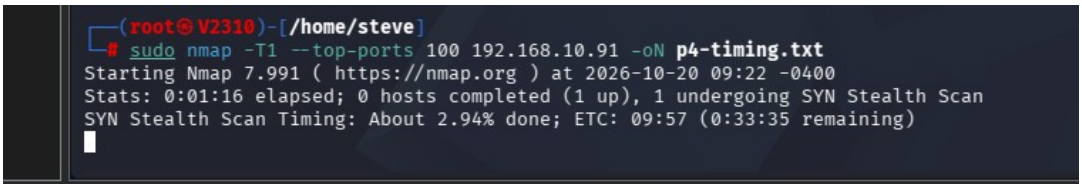


Figure 6 — Step 6: The -T1 slow scan in progress

What this screenshot shows: After 1 minute 16 seconds the -T1 scan was only 2.94% complete, with an estimated 33 minutes 35 seconds remaining for just 100 ports. This proves how extreme the slow-timing technique is — it trades a very long scan time for stealth. The scan was stopped and the timing test was completed with the -T2 (polite) template instead.

The timing test was then completed at -T2 speed on the same top 100 ports:

```
sudo nmap -T2 --top-ports 100 192.168.10.91 -oN p4-timing.txt
```

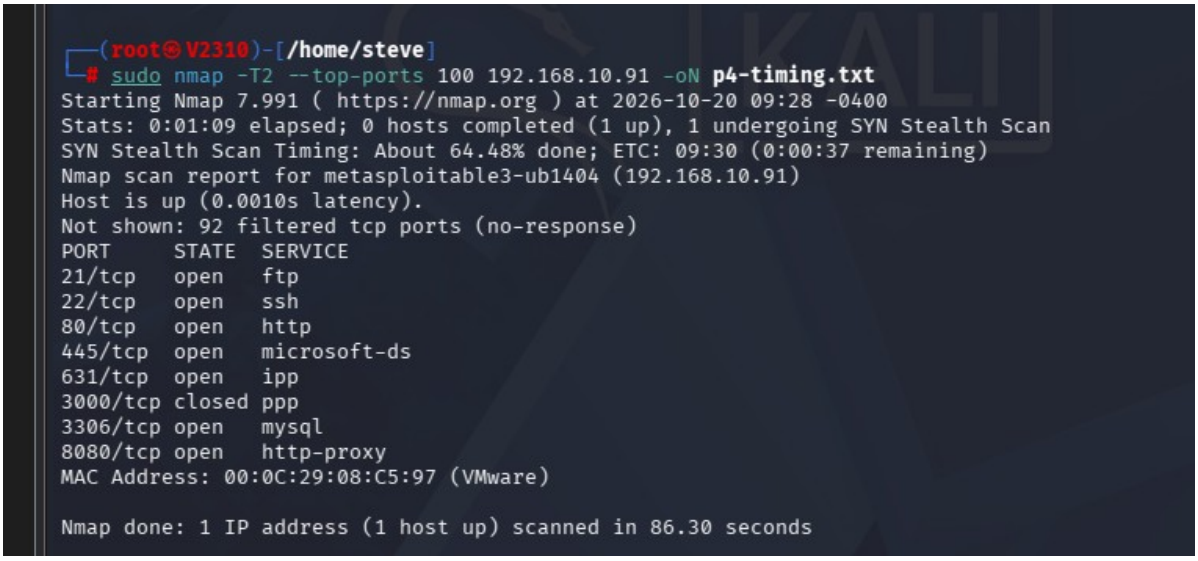


Figure 7 — Step 6: Completed timing scan at -T2 (top 100 ports)

What this screenshot shows: The -T2 scan of the top 100 ports found the same 7 open ports as the baseline (21, 22, 80, 445, 631, 3306, 8080), 1 closed port (3000) and 92 filtered ports — but it took 86.30 seconds, about 16 times longer than the baseline needed for ten times as many ports. Slow timing changed nothing about the results; it only made the scan quieter and much slower.

Step 7 — Add hardening rules to the firewall (on the target).

Three rules were inserted at the top of the INPUT chain, so they are checked before the allow-list. Rule 1 drops fragmented packets. Rule 2 records every source address that opens new TCP connections (recent module, list name SCAN). Rule 3 drops any source that opens more than 20 new connections within 60 seconds — a limit no normal user reaches, but every fast port scan does.

```
sudo iptables -I INPUT 1 -f -j DROP
```

```
sudo iptables -I INPUT 2 -p tcp -m conntrack --ctstate NEW -m recent --name SCAN
--set
```

```
sudo iptables -I INPUT 3 -p tcp -m conntrack --ctstate NEW -m recent --name SCAN
--update --seconds 60 --hitcount 20 -j DROP
```

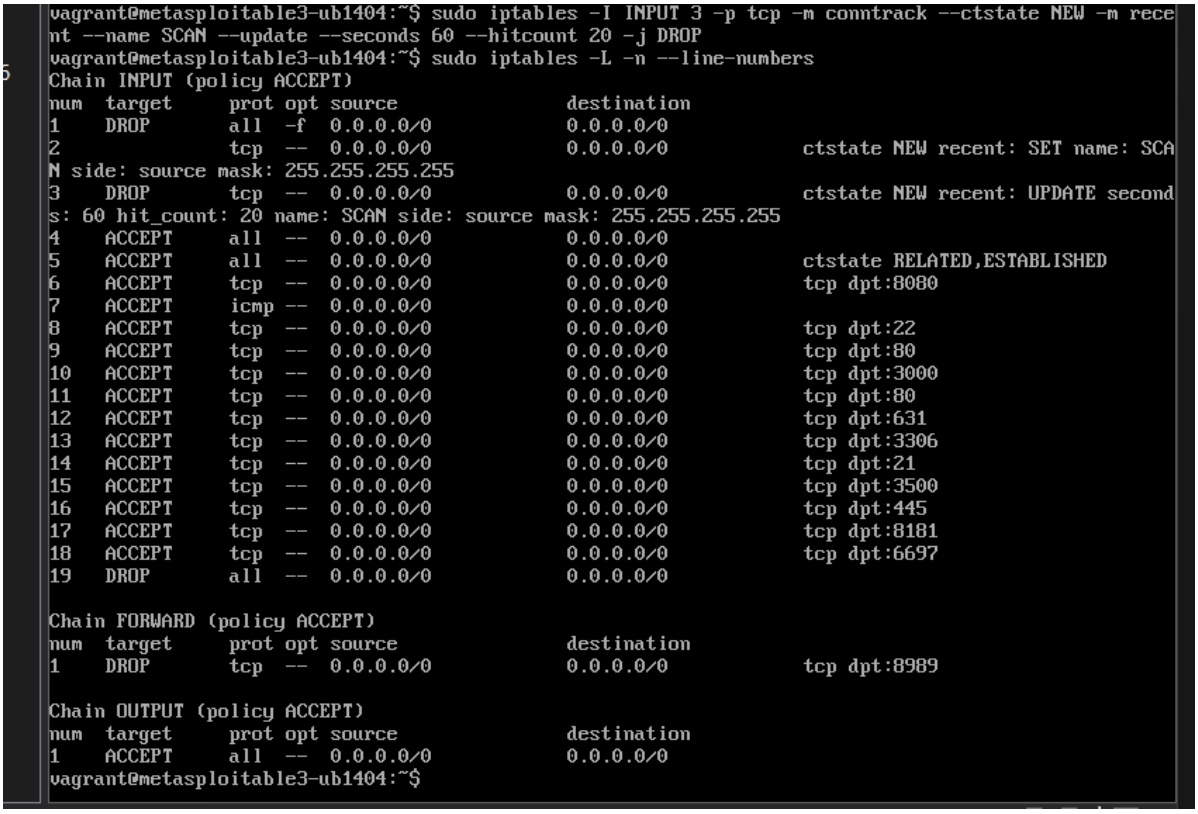


Figure 8 — Step 7: The three new rules at the top of the INPUT chain

What this screenshot shows: The new rules sit at positions 1–3 of the INPUT chain: (1) DROP all fragmented packets (-f), (2) the recent-module SET rule that counts new connections per source, and (3) DROP for sources over 20 new connections in 60 seconds (UPDATE, seconds: 60, hit_count: 20, name: SCAN). The original allow-list rules moved down to positions 4–19, ending with the final DROP. These are the rules written to stop fast scanning and fragmented probes.

Step 8 — Re-test: fast scan against the hardened firewall (on Kali).

The baseline scan was repeated to test the new rules. The full 1,000-port scan was started first, but under the rate limit it stalled — the firewall was dropping nearly all of Kali's probes, so Nmap kept waiting and retrying and the scan did not finish in reasonable time. The re-test was therefore completed on the top 100 ports.

```
nmap --top-ports 100 192.168.10.91 -oN p4-after-rules.txt
```

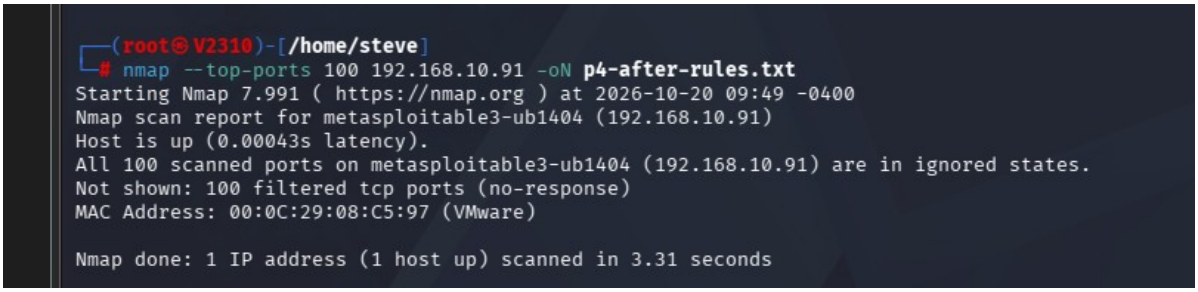


Figure 9 — Step 8: Fast scan after the hardening rules — all ports filtered

What this screenshot shows: Against the hardened firewall, the fast scan found nothing at all: "All 100 scanned ports are in ignored states — 100 filtered tcp ports (no-response)". Before the new rules, the same 100 ports showed 7 open services (Figure 7); after the rules, Kali exceeded 20 new connections almost immediately and its remaining probes were silently dropped, so Nmap received no replies and could not identify a single open port. The student's rules work.

4. Test Matrix

Technique	Nmap Command	Result	Time	Verdict
Baseline (no evasion)	nmap 192.168.10.91	7 open (21, 22, 80, 445, 631, 3306, 8080), 2 closed, 991 filtered	5.34 s	Reference result
Fragmentation	nmap -f	Identical to baseline — no extra ports	5.02 s	Stopped — firewall reassembles fragments
Decoys	nmap -D RND:5	Identical to baseline — no extra ports	4.75 s	Stopped as a bypass — hides the scanner's identity only
Slow timing	nmap -T1, then -T2 --top-ports 100	Same 7 open ports in the top 100	86.30 s (T2); T1 estimated 33:35 more	No bypass — same answers, only slower and quieter
Fast scan after hardening rules	nmap --top-ports 100	0 open — all 100 ports filtered	3.31 s	Blocked by the new rate-limit rules

Reading the matrix: none of the three evasion techniques revealed a single port that the baseline scan had not already shown. Fragmentation and decoys were stopped outright by the stateful, port-based ruleset, and slow timing produced identical results at a much higher time cost. The only technique that changed the outcome was applied by the defender: after the rate-limit rules were added, the fast scan was blocked completely.

5. Ruleset Added by the Student

#	iptables Command	What It Does
1	iptables -I INPUT 1 -f -j DROP	Drops fragmented packets at the door, so fragmented probes can never reach a service
2	iptables -I INPUT 2 -p tcp -m conntrack --ctstate NEW -m recent --name SCAN --set	Counts every new TCP connection from each source address (list: SCAN)
3	iptables -I INPUT 3 -p tcp -m conntrack --ctstate NEW -m recent --name SCAN --update --seconds 60 --hitcount 20 -j DROP	Drops a source that opens more than 20 new connections in 60 seconds — fast scans die, normal use is unaffected

The rules were inserted at the top of the existing INPUT chain (Figure 8), ahead of the target's original allow-list and its final DROP rule, so the tested firewall is the combination of the original ruleset plus these three hardening rules.

6. Observations

- The target was already running a stateful iptables ruleset with a default-drop ending, which is why 991 of the 1,000 scanned ports were filtered even in the baseline scan.
- Fragmentation gained nothing: connection tracking reassembles fragments before the rules are applied, so the firewall judged the same complete packets as in a normal scan.
- Decoys gained nothing against port-based rules: the firewall does not care which address knocks, only which port. Decoys remain useful only for hiding the scanner's identity in logs.
- Slow timing (-T1 / -T2) did not change a single port result. Its only effect is stealth against rate-based detection, and its cost is extreme: -T1 needed an estimated 35 minutes for 100 ports.

- The rate-limit rules were the most effective defence tested: after 20 new connections in a minute, the scanner was dropped entirely, and a repeat fast scan reported all 100 tested ports as filtered.
- A scan slower than the rate limit (fewer than 20 new connections per minute, e.g. -T1) would still slip under Rule 3 — which is why the timing result and the rate-limit result belong together: every defence tested here raises the attacker's cost, but slow and patient scanning remains the hardest to stop.

7. Result

Project 4 is complete. The target's existing iptables firewall was reviewed, a baseline scan recorded 7 open ports, and all three evasion techniques — fragmentation, decoys and slow timing — were tested and recorded in the test matrix: none of them passed the firewall or revealed anything beyond the baseline. Three hardening rules (fragment drop, connection counting, and a 20-per-minute rate limit) were then configured on the target, and the re-test proved them effective: the same fast scan that had found 7 open services now found none — all 100 tested ports filtered. The attacker's tricks were stopped, and the defender's rules were verified by experiment.